Project - Advanced Programming for Biology / Science and Engineering 2026-2027

Version 1.0 - October 12, 2026

David Semedo (df.semedo@fct.unl.pt)

Changelog:

(11/10/2026):

- First version of the project assignment.

1 Introduction

In this project, you will have the opportunity to practice the programming concepts, methodologies, and algorithms covered in the course. Therefore, you are expected to use the approaches and methodologies presented in the lectures and use only the language constructs covered in the course or introduced in the project assignment. **You are not allowed to use any external Python libraries**. If you want to use any other feature not covered in the course or in this assignment, you must previously validate its feasibility with the lecturer.

Project Information:

- The project is to be carried out in a group of two students. Section 6 describes in detail how and when to submit your work.
- The project deadline is on October 26, 23:59:59.
- **Use only base Python.** Everything you need is already part of Python itself: no extra libraries are required, and none should be used (e.g., Biopython, NumPy or pandas).

Overall recommendations:

- The project assignment poses a larger dimension problem compared to the ones done in lab exercises. As such, it is important that you understand the assignment and clarify as soon as possible any questions you may have.
- Commands are presented in increasing order of complexity and dependence. Therefore, we suggest that you implement the different features in the same order as presented in Section 4.
- We will provide tests (on the course Moodle page) so that you can verify the correctness of your implementation. We strongly suggest you use them to debug your program on a step-by-step basis.
- We value the documentation of your source code as well as the usage of the best programming practices, as covered in the course. Please carefully document data classes and functions used to solve the problem, explaining what they mean and what they do, and defining preconditions for their usage, and use type hints in every function.

Project Resources:

- The `data` folder, with the restriction enzymes file, and FASTA files with real sequences from NCBI and small synthetic sequences (will be made available on Moodle).
- Program tests - input and output text files (will be made available on Moodle).

Ethics: Each group project solution must be developed entirely and exclusively by the two group members. The solution to this project should be the product of your effort and commitment. This extends to the use of AI tools such as ChatGPT, Copilot, etc. Nevertheless, students are encouraged to discuss the project with other colleagues, but **must never share their code**.

Code plagiarism is not tolerated. We will use plagiarism detection software to detect such cases. This project is to be conducted by strictly following the Code of Ethics available on Moodle.

Use of AI Assistants Disclaimer: While it is strongly discouraged, in case of using an AI assistant, every function written with the help of an AI assistant (e.g., ChatGPT, Copilot, Claude) must explicitly include the statement `AI-assisted function` in its docstring. This applies to every such function, no matter how much of it was produced by the AI assistant. Be mindful that You are entirely responsible for all the code you submit, and you must be able to explain all of it in the project discussion.

2 Concepts and Objectives

Restriction enzymes are proteins, produced by bacteria, that cut DNA wherever they find a specific short sequence of nucleotides. In nature, bacteria use them as a defence mechanism, chopping up the DNA of invading viruses. In the laboratory, they became one of the most important tools of molecular biology: they are the "molecular scissors" used to map DNA sequences, to check whether a plasmid was correctly assembled, and to cut and paste genes into vectors when cloning. A typical experiment, called a *restriction digest*, consists in cutting a DNA sample with one or more enzymes and then running the resulting fragments on an agarose gel, which separates them by size.

The goal of this project is to develop a program that simulates such experiments *in silico*, i.e., on the computer, assisting a user in planning and interpreting restriction digests. You are asked to develop an interactive program that loads DNA sequences from files, and supports a set of operations over individual sequences or over the whole collection of loaded sequences: from simple statistics, to finding where enzymes cut and simulating a digest.

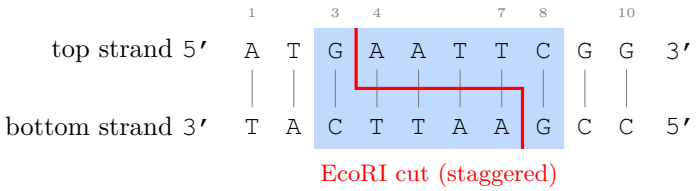


Figure 1: The first 10 base pairs of the sequence `demo`. The top strand is the one written in the FASTA file. The bottom strand is its complement, running in the opposite direction; read $5' \to 3'$, it is `CCGAATTCAT`, the reverse complement of the top strand. EcoRI recognizes `GAATTC` (in blue), which reads the same on both strands, and cuts each strand after the `G`: between positions 3 and 4 on the top strand, and between positions 7 and 8 on the bottom strand.

2.1 DNA Strands

A DNA molecule is made of two **strands**, wound around each other in the well-known double helix. Each strand is a long chain of *nucleotides*, and each nucleotide carries one of four *bases*: `A` (adenine), `C` (cytosine), `G` (guanine) and `T` (thymine). A DNA sequence is thus represented as a string over the alphabet `A`, `C`, `G`, `T`. The two strands are held together by hydrogen bonds between pairs of bases, following two rules (Figure 1):

Complementarity - Each base only pairs with its *complementary* base: `A` pairs with `T`, and `C` pairs with `G`. Each such pair is a *base pair* (bp), which is why the length of DNA is measured in base pairs.

Antiparallelism - Each strand has a direction, which comes from the chemical structure of its backbone: one end is called the $5'$ end (read "five prime") and the other the $3'$ end. The two strands run in *opposite* directions: where one strand goes $5' \to 3'$, the other goes $3' \to 5'$.

By convention, a strand is always read and written in the $5' \to 3'$ direction, which is also the direction in which cells read and copy DNA. Given one strand, the other one is fully determined: it

is its **reverse complement**, i.e., the complement of each base, read in reverse order. For example, the reverse complement of `ATGAATTCGG` is `CCGAATTCAT`.

Since one strand determines the other, databases and files store only one of them, called the **top strand** (also called forward, or plus, strand). The other strand is the **bottom strand** (reverse, or minus, strand). In this project, **all operations are done over the top strand**, exactly as written in the files, and all positions refer to the top strand.

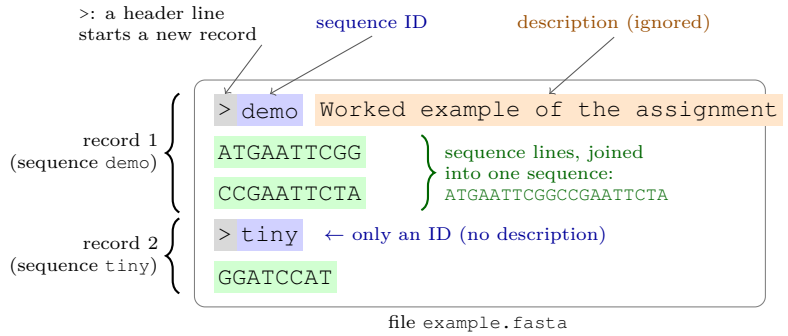


Figure 2: The parts of a FASTA file, in an illustrative file with two records (lines were kept short here; real files usually have 60 or 70 bases per line). Each record starts with a header line, beginning with >, followed by one or more sequence lines, until the next header line or the end of the file.

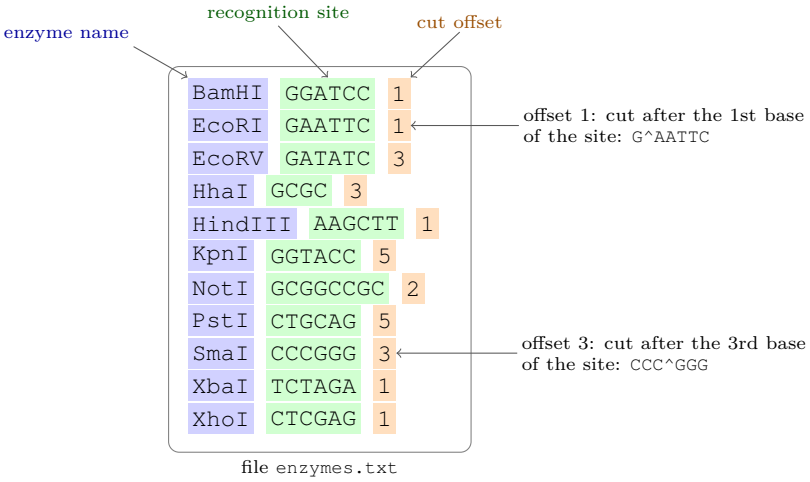


Figure 3: The restriction enzymes file. Each line describes one enzyme, with three fields separated by a space: its name, its recognition site and its cut offset.

2.2 FASTA Files

The FASTA format [1] is the most widely used text format to store biological sequences. Figure 2 shows the parts of a FASTA file. A FASTA file contains one or more *records*, and each record has:

Header line - A line starting with the > character. The text right after >, up to the first space (or the end of the line), is the **sequence identifier** (ID). Anything after the first space is a free-text description, which your program should ignore.

Sequence lines - One or more lines with the sequence itself. Long sequences are split across several lines (usually of 60 or 70 characters, but any length is possible), which must be joined to obtain the complete sequence.

Positions in a sequence are **1-based**, as is standard in biology: the first base of a sequence is at position 1 (careful: in Python, it is at index 0!).

2.3 Restriction Enzymes

Each restriction enzyme recognizes a specific, short sequence, its **recognition site**, and cuts the DNA at a fixed place within it. For example, the enzyme EcoRI recognizes the site `GAATTC` and

cuts it between the `G` and the first `A`, which biologists write as `G^AATTC`. We describe where an enzyme cuts with an integer, the **cut offset**: the number of bases of the site that come before the cut. The cut offset of EcoRI is thus 1, while the one of SmaI (`CCC^GGG`) is 3. The cut offset of an enzyme is always greater than 0 and smaller than the length of its site, i.e., enzymes always cut inside their site.

The enzymes supported by the program are described in the file `enzymes.txt` (Figure 3), with one enzyme per line, in the format: `name site offset`. You can find information about these, and thousands of other enzymes, in the REBASE database [4].

Note: Recognition sites are usually *palindromic*: the reverse complement of `GAATTC` is `GAATTC` itself. Therefore, the site reads the same on both strands (section 2.1), which is why it is enough to search for sites on the top strand. Enzymes cut *both* strands, each one at the same place of the site, read $5' \to 3'$. As the strands run in opposite directions, the two cuts are usually staggered (Figure 1), leaving short single-stranded ends, called *sticky ends*, which are what allows fragments cut by the same enzyme to be pasted together when cloning. In this project, we only consider the cut on the top strand: the cut offset of each enzyme refers to the top strand.

2.4 Restriction Digests

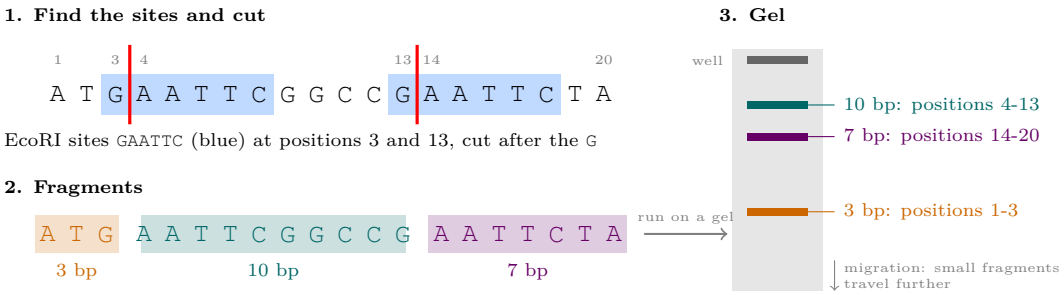


Figure 4: Digestion of the sequence `demo` (20 bp) with EcoRI. (1) EcoRI recognizes two sites and cuts each one after its first base. (2) The two cuts split the sequence into three fragments, each one shown in its own color. (3) In a gel, the fragments are separated by size: each band is a fragment, and the larger the fragment, the closer it stays to the well. The command DIGEST lists the fragments in this same order, from the top to the bottom of the gel.

To simulate a digest of a sequence s, we find all occurrences of the recognition sites of the enzymes in s. Each occurrence starting at position p (1-based), of an enzyme with cut offset o, cuts s between the positions $p + o - 1$ and $p + o$. A linear sequence with n distinct cuts is thus split into $n + 1$ fragments.

Figure 4 shows the digestion of the sequence `demo`, `ATGAATTCGGCCGAATTCTA` (20 bp, base pairs), with EcoRI. The site `GAATTC` occurs at positions 3 and 13, so the sequence is cut between positions 3 and 4, and between positions 13 and 14. This produces three fragments: `ATG` (positions 1-3, 3 bp), `AATTCGGCCG` (positions 4-13, 10 bp) and `AATTCTA` (positions 14-20, 7 bp). Note that the lengths of the fragments always add up to the length of the sequence.

Note that occurrences of a site may overlap. For example, in the sequence `GCGCGC`, the site `GCGC` of the enzyme HhaI occurs twice, at positions 1 and 3, and both must be found.

2.5 Your Project Directory Structure - Code + Data Files

You will be reading files from a folder named `data`, which you can download from Moodle. It contains the enzymes file and all FASTA files. This folder should be in the same folder as your `main.py` file.

Besides small synthetic sequences, made for testing specific situations (e.g., `demo.fasta`, `constructs.fasta` and `plasmid.fasta`), the folder contains real sequences, downloaded from the NCBI Nucleotide database [2], listed in Table 1. Their IDs are NCBI accession numbers, which include a version number after a dot (e.g., `L09137.2`). In practice, your project directory structure should look like this:

```
/
├── main.py
└── data/
```

```
├── enzymes.txt
├── demo.fasta
├── constructs.fasta
├── ...
├── pUC19.fasta
└── ...
```

File	Sequence ID(s)	Description	Length (bp)
pUC19.fasta	L09137.2	Cloning vector pUC19 (circular)	2686
pBR322.fasta	J01749.1	Cloning vector pBR322 (circular)	4361
M13mp18.fasta	X02513.1	Cloning vector M13mp18 (circular)	7249
lambda.fasta	J02459.1	Bacteriophage lambda, complete genome	48502
phiX174.fasta	NC_001422.1	Bacteriophage phiX174, complete genome (circular)	5386
SV40.fasta	NC_001669.1	Simian virus 40, complete genome (circular)	5243
globins.fasta	NM_000518.5	Human hemoglobin subunit beta (HBB) mRNA	628
	NM_000558.5	Human hemoglobin subunit alpha 1 (HBA1) mRNA	577
	NM_000519.4	Human hemoglobin subunit delta (HBD) mRNA	620

Table 1: Real sequences available in the data folder, from the NCBI Nucleotide database.

Reading Files. To avoid issues with files on different Operating Systems (CodeGrade runs on Linux, you might be running your code on Windows/macOS), you should use the Python function os.path.join to create each file path. Namely, given a file name provided by the user (e.g. demo.fasta), you should create the file path as:

```
#At the top of your script file
import os

#Define a constant with the folder name
DATA_FOLDER = "data"

#Inside the function that will be reading a file
file_path = os.path.join(DATA_FOLDER, "demo.fasta")
```

Then, the variable file_path will have the path that you should provide to the open function.

2.6 Visualizing Digests

It is very convenient to be able to visualize sequences and digests, to check your results. We suggest that you use NEBcutter [3]: you can paste any sequence, see where each enzyme cuts and simulate digests. Keep in mind that NEBcutter shows positions of the cuts, while some commands of this project show positions of the sites, and that by default, it includes many more enzymes than the ones in our enzymes.txt file.

2.7 Program Overview

Figure 5 gives an overview of what your program should do. When it starts, it reads the enzymes file. Then, it works as a command interpreter: it repeatedly asks the user for a command, executes it, and prints the corresponding messages, until the user types EXIT. Its features, provided by the commands detailed in Section 4, can be organized in three groups:

1. **Load** - read FASTA files from the data folder, validating them (LOAD_SEQUENCES);
2. **Inspect** - list the loaded sequences and show information about each one (LIST_SEQUENCES and SEQUENCE_INFO);
3. **Analyse** - use the enzymes to find recognition sites and simulate digests (FIND_SITES and DIGEST).

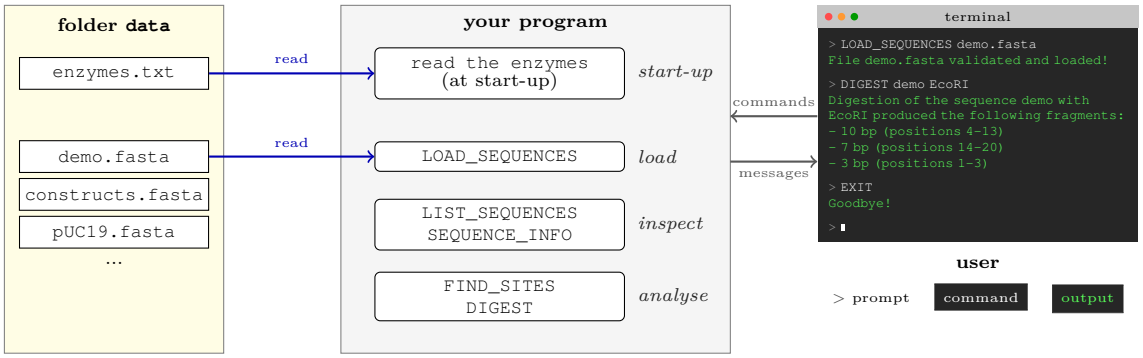


Figure 5: Overview of the program. At start-up, it reads the enzymes file. Then, it repeatedly reads a command typed by the user and prints the corresponding output. Its features allow to *load* FASTA files from the `data` folder, *inspect* the loaded sequences, and *analyse* them with the enzymes. In the terminal, > represents the prompt (`Command to execute:`, in your program), commands typed by the user are shown in white, and the output of the program in green.

3 Structuring Your Solution

Before writing any code, read the whole assignment and think about how to structure your program, using **data classes** and functions, as covered in the course (lecture *L08 - Data Classes*). A well-structured solution is easier to write, test, debug and **explain**, and also easier to **extend** later, with new features. Before you start, try to answer the following questions:

1. Which concepts of the problem should the program represent, and which of them deserve a data class?
2. Which fields does each data class need, and which data can be computed instead?
3. Which functions work on each data class, and what does each one receive and return?
4. How are the data classes related, and which data structures should hold them?
5. Which parts of the program read input and print output, and which parts only compute results?
6. If a new feature or a new command had to be added, how many parts of your program would need to change?

Be prepared to discuss these design decisions in the project discussion.

Code practices. Your code should also follow the practices covered in the course:

- **A single file**, `main.py`, with all your code, organized in sections, in this order: imports, constants, data classes, domain functions, `_io` functions, the `main` function, and the `if __name__ == "__main__":` guard.
- **Type hints** in every function: the types of its parameters and of its return value, including `-> None`.
- **Documentation**: a docstring for each data class, and for each function whose purpose is not obvious from its name, with its preconditions, if any; comments explaining the less obvious parts of the code.
- **Constants** in `UPPER_SNAKE_CASE`: no unexplained numbers or repeated strings (e.g., messages) inside function bodies.
- Functions that compute results (domain functions) never call `input()` or `print()`; one `_io` function per command; a short `main` function, called only through the `if __name__ == "__main__":` guard.
- **Data classes** (`@dataclass`) to represent the concepts of the problem, as covered in the course: data classes only hold data, in their fields, read and changed as `object.field`, and the operations on your data are functions, which receive data classes as parameters.
- Names that say what each function does or returns.

4 Functionalities

The next sections describe each of the individual features that your program should support and provide use-case examples of the corresponding commands.

In all situations, you can assume that the input provided by the user will always be correct and consistent with the command definition, i.e., if a command receives a number, you can assume that the user will never provide a string. You can also assume that every FASTA file provided by the user exists and is correctly formatted (it may, however, contain invalid nucleotides, see command LOAD_SEQUENCES), and that every sequence has at least one base.

Program Flow: When your program starts, it should read the enzymes file, `enzymes.txt`, which is always present in the `data` folder and always valid. Then, the command LOAD_SEQUENCES can be used to load the sequences of a FASTA file. All commands can be invoked at any time, in particular, the user should be able to load another file at any time. The command EXIT terminates the execution.

Command prompt: Your command interpreter should always print the following prompt message, and wait for an user command:

```
Command to execute:
```

Unknown Commands: Every time the user provides an unknown command, the program should output the following message:

```
Unknown command.
```

Common error messages: Many commands receive a sequence ID and/or enzyme names. Whenever the sequence ID does not correspond to a loaded sequence, or an enzyme name does not exist in the enzymes file, the program should print, respectively:

```
The sequence <id> is not loaded.
The enzyme <name> is not available.
```

Unless stated otherwise, commands first check the sequence ID, and only then the enzyme names (in the order they were given, reporting only the first unknown one). Command names, sequence IDs and enzyme names are case-sensitive: for example, `digest` is an unknown command, and `ecori` is not the enzyme EcoRI.

4.1 Terminating the Program

Command EXIT: This command terminates the execution of the program.

- Example of an execution:

```
EXIT
Goodbye!
```

4.2 Loading and Verifying a FASTA File

Command LOAD_SEQUENCES: This command takes the name of a FASTA file as input (e.g. `constructs.fasta`) and loads all of its sequences, by first validating its integrity. In particular, it should check the following:

1. The provided file hasn't been loaded yet.
2. All sequences of the file contain only valid nucleotides: `A`, `C`, `G` and `T` (uppercase).
3. Sequence IDs are unique: no sequence of the file has the same ID as an already loaded sequence, or as a previous sequence of the same file.

If the file integrity is not valid, then the program should provide an error message indicating what was the cause. Your program should check each of the three situations in the same order as they were listed above. As soon as one of the checks fails, it should stop the validation and present a message to the user. Checks 2 and 3 go through the sequences in the order they appear in the file, and report the first sequence that fails. If any check fails, **no sequence of the file is loaded**, and the file is not considered loaded: loading it again repeats the validation.

- Example of a program execution with a valid FASTA file:

```
LOAD_SEQUENCES constructs.fasta
File constructs.fasta validated and loaded!
```

- Example of a program execution of loading an already loaded file (1st validation step):

```
LOAD_SEQUENCES demo.fasta
File demo.fasta already loaded!
```

- Validation error on the 2nd validation step:

```
LOAD_SEQUENCES bad_bases.fasta
File bad_bases.fasta could not be loaded. Sequence sampleB contains invalid nucleotides.
```

- Validation error on the 3rd validation step (with `constructs.fasta` already loaded):

```
LOAD_SEQUENCES duplicate_ids.fasta
File duplicate_ids.fasta could not be loaded. Sequence ID pLac_frag is already in use.
```

4.3 List All Loaded Sequences

Command LIST_SEQUENCES: This command lists all currently loaded sequences, sorted alphabetically by ID, in ascending order[1]. It should distinguish two situations: 1) when at least one sequence is loaded, it prints the number of sequences loaded and lists all sequences, with their length, according to the format below, 2) otherwise, it tells the user that there are no loaded sequences.

- Example of the LIST_SEQUENCES command, after loading the files `demo.fasta` and `constructs.fasta`:

```
LIST_SEQUENCES
Number of sequences loaded: 4
- amp_cassette (150 bp)
- demo (20 bp)
- pGFP_insert (240 bp)
- pLac_frag (180 bp)
```

- Example of LIST_SEQUENCES command when no sequences are loaded:

```
LIST_SEQUENCES
No sequences loaded.
```

4.4 Sequence Information

Command SEQUENCE_INFO: This command presents basic information about a loaded sequence: its length, the number of occurrences of each base, and its GC content. The GC content is the percentage of bases that are `G` or `C`, rounded to 2 decimal places with the function `round(gc, 2)`, and printed as returned by it, i.e., without trailing zeros (e.g., `40.0`, `47.5` or `56.25`). GC content is an important property of DNA: for example, GC-rich regions are more stable, as G-C pairs are held by three hydrogen bonds, against two in A-T pairs.

- Example of an execution for the `demo` sequence:

[1] As done by Python's `sorted` function over strings.

```
SEQUENCE_INFO demo
Information about the sequence demo:
- Length: 20 bp.
- Base counts: A=6, C=4, G=4, T=6.
- GC content: 40.0%.
```

- Example of an execution for the `pGFP_insert` sequence:

```
SEQUENCE_INFO pGFP_insert
Information about the sequence pGFP_insert:
- Length: 240 bp.
- Base counts: A=73, C=63, G=51, T=53.
- GC content: 47.5%.
```

- Example of an execution in which the sequence is not loaded:

```
SEQUENCE_INFO sampleA
The sequence sampleA is not loaded.
```

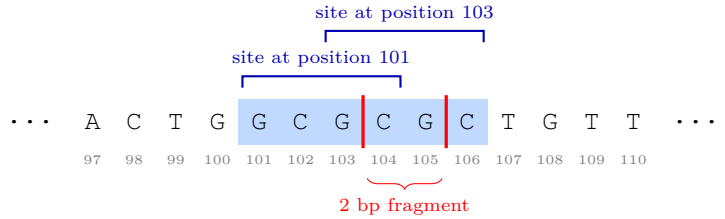


Figure 6: Overlapping recognition sites of HhaI (`GCGC`, cut as `GCG^C`) in the sequence `pLac_frag`. The bases `GCGCGC` (positions 101 to 106) contain two occurrences of the site, which share the bases at positions 103 and 104. Both must be found: after finding a site at position p, the search must continue from position $p+1$, not from the end of the site. The two cuts (in red) produce a fragment with only 2 bp.

4.5 Finding Recognition Sites

Command FIND_SITES: This command finds all the recognition sites of a given enzyme in a given sequence. It presents the number of sites found and their positions, i.e., the (1-based) position of the first base of each occurrence of the site, in ascending order. Remember that occurrences of a site may overlap (section 2.4), and that all of them must be found.

The command fails in three situations: 1) the sequence is not loaded, 2) the enzyme is not available, and 3) the enzyme has no recognition site in the sequence. Figure 6 shows why the search for the next site must start right after the beginning of the previous one.

- Example of an execution for the `demo` sequence and EcoRI (Figure 4):

```
FIND_SITES demo EcoRI
Recognition sites of EcoRI (GAATTC) in the sequence demo:
- Number of sites: 2.
- Positions: 3, 13.
```

- Example of an execution with overlapping sites (`pLac_frag` contains `GCGCGC`, starting at position 101):

```
FIND_SITES pLac_frag HhaI
Recognition sites of HhaI (GCGC) in the sequence pLac_frag:
- Number of sites: 2.
- Positions: 101, 103.
```

- Example of an execution in which the enzyme has no site in the sequence:

```
FIND_SITES pLac_frag BamHI
The enzyme BamHI does not cut the sequence pLac_frag.
```

- Example of an execution with an enzyme that is not available:

```
FIND_SITES pLac_frag EcoRX
The enzyme EcoRX is not available.
```

4.6 Simulating a Digest

Command DIGEST: This command simulates the digestion of a (linear) sequence with one or more enzymes, as described in section 2.4, and lists the resulting fragments, as they would appear in an agarose gel: **sorted by length, in descending order**, and in case of a tie, by start position, in ascending order. For each fragment, the program presents its length and its start and end positions (1-based, inclusive) in the original sequence.

To execute this command, the user provides a sequence ID followed by one or more enzyme names. When several enzymes are used, the cuts of all of them are combined. If two enzymes cut at exactly the same place (or an enzyme is given twice), that cut is only counted once. The first line of the output lists the enzyme names exactly as given by the user, in the same order, separated by a comma and a space (repeated names included).

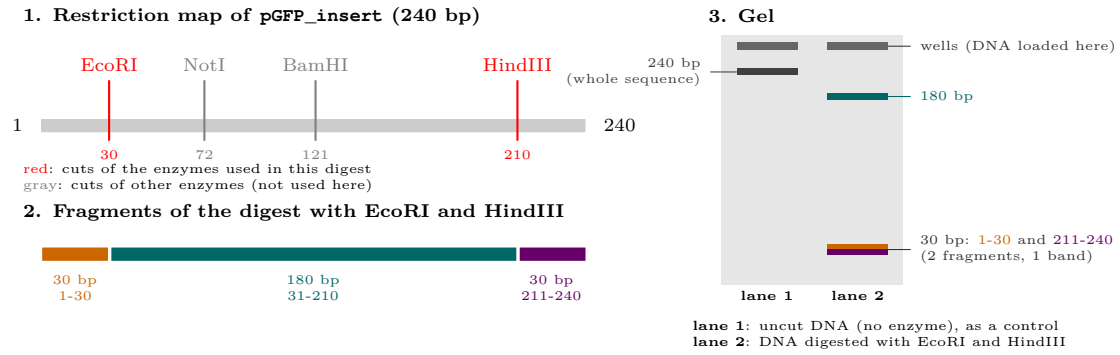


Figure 7: Digestion of the sequence `pGFP_insert` with EcoRI and HindIII. (1) The restriction map shows where each enzyme cuts the sequence. (2) The two cuts used produce three fragments, each one in its own color. (3) In the gel, lane 1 is a control: the same DNA, not digested, which forms a single band with the length of the whole sequence. Lane 2 is the digest: fragments with the same length travel the same distance, so the two 30 bp fragments form a single band. In the output of the command DIGEST, they are listed by start position.

The command fails in three situations: 1) the sequence is not loaded, 2) one of the enzymes is not available, and 3) none of the enzymes cuts the sequence. Figure 7 shows a digest with two enzymes, and the corresponding gel.

Note on sorting: To sort by multiple criteria while handling ties, *custom sorting* is needed. This can be done with the built-in `sorted()` function or the `.sort()` method, by providing a function that specifies the sorting criterion. For example:

```
# List of fragments, as tuples: (start, end, length)
fragments = [(1, 30, 30), (31, 210, 180), (211, 240, 30)]

# Sort by length (descending), then by start position (ascending)
gel = sorted(fragments, key=lambda frag: (-frag[2], frag[0]))
```

The key `-frag[2]` sorts the lengths in **descending** order, and `frag[0]` breaks ties by start position, in ascending order.

- Example of an execution for the `demo` sequence with EcoRI (Figure 4):

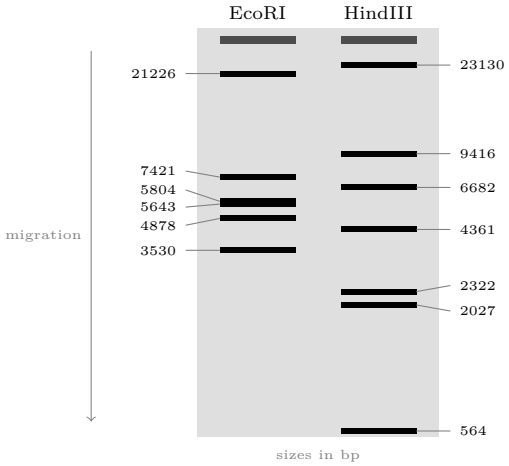


Figure 8: Simulated gel of the genome of bacteriophage lambda (48 502 bp), digested with EcoRI and with HindIII, using the fragment sizes computed by the command DIGEST. Distances are proportional to the logarithm of the size, as in a real gel. Lambda digests are classic *size markers*: their bands have well-known sizes, so running them next to an unknown sample allows estimating the sizes of its fragments. Note that the EcoRI fragments of 5804 and 5643 bp are so close in size that they almost form a single band.

```
DIGEST demo EcoRI
Digestion of the sequence demo with EcoRI produced the following fragments:
- 10 bp (positions 4-13)
- 7 bp (positions 14-20)
- 3 bp (positions 1-3)
```

- Example of an execution with two enzymes, and two fragments with the same length:

```
DIGEST pGFP_insert EcoRI HindIII
Digestion of the sequence pGFP_insert with EcoRI, HindIII produced the following fragments:
- 180 bp (positions 31-210)
- 30 bp (positions 1-30)
- 30 bp (positions 211-240)
```

- Example of an execution with three enzymes:

```
DIGEST pLac_frag EcoRI HhaI XhoI
Digestion of the sequence pLac_frag with EcoRI, HhaI, XhoI produced the following fragments:
- 55 bp (positions 17-71)
- 39 bp (positions 142-180)
- 36 bp (positions 106-141)
- 32 bp (positions 72-103)
- 16 bp (positions 1-16)
- 2 bp (positions 104-105)
```

- Example of an execution for the genome of bacteriophage lambda (file `lambda.fasta`). Lambda DNA digested with EcoRI is a classic *size marker*: a set of fragments of well-known sizes, run next to unknown samples on a gel to estimate their sizes (Figure 8).

```
DIGEST J02459.1 EcoRI
Digestion of the sequence J02459.1 with EcoRI produced the following fragments:
- 21226 bp (positions 1-21226)
- 7421 bp (positions 31748-39168)
- 5804 bp (positions 39169-44972)
- 5643 bp (positions 26105-31747)
- 4878 bp (positions 21227-26104)
- 3530 bp (positions 44973-48502)
```

- Example of an execution in which none of the enzymes cuts the sequence:

```
DIGEST pLac_frag BamHI
The sequence pLac_frag is not cut by the provided enzymes.
```

- Example of an execution with an enzyme that is not available:

```
DIGEST pLac_frag EcoRI EcoRX
The enzyme EcoRX is not available.
```

5 Evaluation Criteria

The project evaluation comprises two main criteria:

Functionality and Correctness (12 values) - Which features were implemented and the correctness of each of those features, measured by the automatic tests of CodeGrade, which score your submission from 0 to 100 points. Full grade corresponds to a CodeGrade score of 100, i.e., if your submission got 75 points on CodeGrade, then, your solution is graded with 9 values for this criteria. A solution with all the commands implemented correctly will have a CodeGrade score of 100 points.

Code quality (8 values) - Quality of the code according to the principles covered and taught in the course (e.g., code structure, chosen data classes, clarity, comments and docstrings, type hints, etc.), as described in Section 3.

The grading of each part is independent and does not affect each other (i.e., the Code Quality grade cannot decrease the functionality, nor vice versa). For example, if you get 75 points in CodeGrade and 8.5 values in the Code Quality component, your grade for the project will be $9 + 8.5 = 17.5$ values.

The 100 points of CodeGrade are associated with the commands of the project, and are distributed equally across 20 tests (5 points each), which introduce the commands progressively, in the order of Section 4. Each test checks a sequence of commands, and is only passed if the whole output of your program matches the expected output. The tests we provide on Moodle are the ones used by CodeGrade.

Project discussion. The project discussion of the work is mandatory and may take the form of a written or/and oral discussion. Discussions will focus on your final submitted code. You are expected to know and be capable of explaining all the code submitted.

Some groups might be excluded from the project discussion. The grade of each group member depends on the grade of the work and the individual performance in the discussion. Consequently, the grades of the two elements of the group may be different.

6 Project Submission

Where to submit: To submit your solution to this project assignment, you should submit your code directly on CodeGrade (not on Moodle), in the project assignment. The submission consists of a single file, `main.py`, with all your code (Section 3). You do not need to submit the `data` folder: CodeGrade provides its own copy of it.

Project Deadline: October 26, 23:59:59.

How to submit: The project is submitted as a group, in CodeGrade:

1. One of the group members creates a group, under the project assignment, in CodeGrade.
2. The other group member joins that group.
3. Submissions are made in the name of the group, and can be made by either of the two group members.

The group must be created before the first submission. For a step-by-step guide on how to create and join a group, see:

https://help.codegrade.com/for-students/advanced-features/handing-in-as-a-group

After each submission, CodeGrade runs the automatic tests and shows you their results. You can re-submit the work as often as you like, up to the submission deadline. **Only the last submission with the highest score will be evaluated**; if there are multiple submissions with the highest score, among these, the last that has been submitted will be evaluated.

Before submitting: Remember to comment and document your code, and to add type hints, if you have not done so yet: each data class, and each function whose purpose is not obvious from its name, should have a docstring explaining what it does and its preconditions, every function should have type hints, and less obvious parts of the code should be explained with comments. These are part of the Code quality criterion of the evaluation (Section 3).

References

[1] National Center for Biotechnology Information. FASTA Format. https://www.ncbi.nlm.nih.gov/genbank/fastaformat/.

[2] National Center for Biotechnology Information. Nucleotide Database. https://www.ncbi.nlm.nih.gov/nucleotide/.

[3] New England Biolabs. NEBcutter – Analyze DNA Sequences with Restriction Enzymes. https://nc3.neb.com/NEBcutter/.

[4] New England Biolabs. REBASE – The Restriction Enzyme Database. http://rebase.neb.com.